零安装隧道即服务：面向个人计算设备的极简远程访问系统设计与分析

mobile ai: Zero-Install Tunnel-as-a-Service for Minimal Remote Access to Personal Computing Devices — Design and Analysis

系统版本基线：v1.0（2026-09） 目标读者：系统安全、边缘计算与个人计算方向研究者
关键词：隧道即服务；零安装远程访问；能力安全（capability security）；URL-as-credential；零知识控制面

摘要

【中文】远程访问个人计算设备长期被两类方案割裂：一类（Tailscale、ZeroTier）要求两端安装客户端并建立虚拟私有网络，部署摩擦大；另一类（ngrok、cloudflared quick tunnel）只需一端安装但缺乏身份管理与撤销机制，随机 URL 泄露即失守。本文提出并分析 **mobile ai**——一个「零安装的隧道即服务」系统：用户在家中电脑上执行一条 `curl | bash` 命令，经 Cloudflare Tunnel 建立出站连接，获得形如 `https://<随机子域>.newapi.email` 的专属 URL；手机浏览器直达家中电脑的本地服务（默认 DSH Web GUI），全程无需安装任何移动端应用、无需公网 IP 与端口转发。v1.0 版本进一步简化了身份模型：取消一次性认证码，以机器码（machine code）作为唯一身份标识实现无码自动注册；引入本地多路复用器（mux）支持单隧道挂载多个工具服务；提供工具预设与三平台一键安装脚本。系统采用严格分层的架构：控制面（Node.js + SQLite）仅负责机器码注册、绑定管理、心跳校验与在线时长追踪；数据面完全位于用户机器，Cloudflare 边缘仅做 TLS 终结与转发，平台不解密、不代理业务流量。安全模型以能力（capability）范式为核心：随机子域 URL 即访问凭证，URL 轮换等价于撤销（revocation），配合机器绑定心跳形成纵深。本文给出该模型的形式化论证（零留存、撤销语义），基于 Cloudflare 免费计划公开限制做保守容量估算，并以 trade-off 表评估各设计决策。系统已在生产环境运行（v1.0）。

(Abstract, English) Remote access to personal computing devices has been split between two camps: client-heavy approaches such as Tailscale and ZeroTier, which require installing software on *both* ends to form a virtual private network; and one-sided tunneling tools such as ngrok or Cloudflare quick tunnels, which require no client on the consumer device but provide little in identity management and revocation. We present **mobile ai**, a *zero-install tunnel-as-a-service*: the user runs a single `curl | bash` command on their home computer, establishing an outbound Cloudflare Tunnel that yields a dedicated URL of the form `https://<random-subdomain>.newapi.email`; a phone browser reaches local services on the home machine (by default, the DSH Web GUI) with no mobile app and no public IP or port forwarding. Version 1.0 further simplifies the identity model: the one-time authentication code is eliminated, with a machine code serving as the sole identifier for automatic no-code registration; a local multiplexer (mux) supports mounting multiple tool services on a single tunnel; and three-platform one-click install scripts are provided. The system is strictly layered: the control plane (Node.js + SQLite) handles only machine-code registration, binding management, heartbeat liveness, and online-time tracking; the data plane lives entirely on the user's machine, with Cloudflare edge performing TLS termination and forwarding only — the platform never decrypts or proxies business traffic. The security model is capability-based: a random subdomain URL *is* the access credential, and URL rotation equals revocation. We formalize this model (zero-retention and revocation semantics), give conservative capacity estimates from Cloudflare's public

free-plan limits, and evaluate each design decision via a trade-off table. The system is in production (v1.0).

1 引言（Introduction）

「人在路上，家里的电脑却闲置」是个人计算时代一个朴素而普遍的问题。随着 AI 编码代理（如 DeepSeek Harness、Codex）与自托管 Web 应用日益依赖长时运行的本地算力，用户希望在手机浏览器中直接操作家中电脑的完整工具链——而非远程桌面那样转发屏幕画面，而是直达本地 HTTP 服务本身。

现有方案存在三类割裂：

1. **端到端客户端 VPN（Tailscale、ZeroTier）**[1][2]。两端均需安装并登录同一虚拟网络，部署摩擦大，且引入一个持续运行的 VPN 平面，对个人用户而言是「为了访问一台机器而改造整个网络」。
2. **通用隧道工具（ngrok、cloudflared quick tunnel）**[3][4]。只需一端安装，但 ngrok 的免费隧道 URL 短且需账号绑定；cloudflared quick tunnel（`--url https://trycloudflare.com`）生成的随机 URL 无任何身份管理，泄露即失守、无法撤销。
3. **SaaS 托管（如各类 cloud IDE）**。数据与算力上云，违背「一切留在自己机器」的隐私诉求。

mobile aiv1.0 的设计目标是同时满足五个看似冲突的属性：(a) 用户侧零安装（一条命令 + 一个浏览器页面）；(b) 消费端零应用（手机只用系统浏览器）；(c) 平台侧零业务流量接触（控制面不解密、不留存）；(d) 可撤销的访问凭证；(e) 无码自动认证（machine code = identity，无需一次性码）。这五个属性在现有方案中从未同时出现：客户端 VPN 牺牲了 (a)(b)，隧道工具牺牲了 (d)(e)（或仅有平台侧、不可用户自助的撤销），SaaS 托管牺牲了 (c)。本文论证：只要把「身份与撤销」从数据路径中剥离到独立的控制面，并把多路复用下沉到用户机器本地，五者即可共存。本文的贡献：

- **架构**：控制面/数据面严格分离的分层设计，本地轻量 mux 实现「一条隧道、多工具」；v1.0 引入无码自动注册（machine code = identity）与工具预设系统；
- **安全模型**：以能力范式（capability-based security）[5] 论证「URL-as-credential」与「轮换 = 撤销」的语义，并给出零留存（zero-retention）论证；
- **工程分析**：基于公开免费配额的保守容量估算，以及面向设计决策的 trade-off 评估表（含 v1.0 无码认证 vs v0.5 一次性码的对比）。

2 相关工作（Related Work）

2.1 端到端客户端 VPN

Tailscale [1] 基于 WireGuard，以「mesh over DERP」提供端到端加密的点对点连接，强调零配置与设备身份；ZeroTier [2] 提供虚拟局域网语义。两者都要求**两端安装客户端并加入同一命名网络**。其优势是设备身份与 mDNS 发现，代价是每个端点都要一个常驻 agent。mobile ai 的差异化在于**消费端零安装**：手机只是一个 HTTPS 客户端，不加入任何虚拟网络。

2.2 隧道与端口暴露工具

ngrok [3] 提供带身份与审计的隧道，但客户端需注册并绑定账号，URL 生命周期由平台控制；Cloudflare Tunnel（cloudflared）[4] 支持命名隧道与 token 模式，后者**本地无 `config.yml`、ingress 完全由控**

制面经 API 掌控——这是 mobile ai「一个隧道挂多个 ingress」可行的前提。Cloudflare quick tunnel 则不提供持久身份与撤销，mobile ai 在其之上补全了**一次性码 + 机器绑定 + 轮换撤销**。

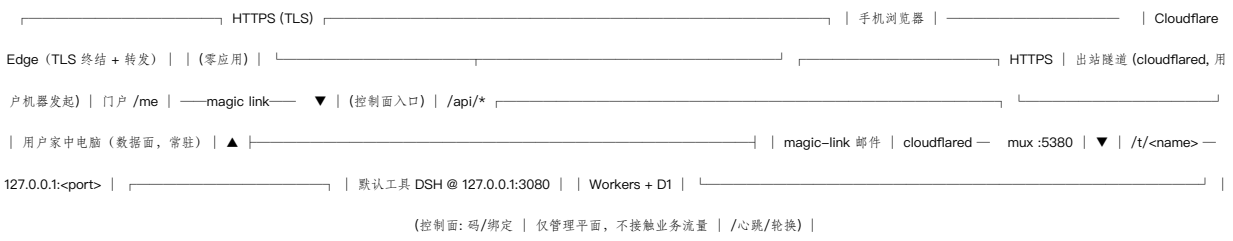
2.3 凭据与访问控制理论

- **能力安全（capability security）**[5][6]：一个不可伪造的引用（token）本身即授权，持有即可访问、可传递；撤销通过**失效该引用或使其不可再被解析**。mobile ai 的随机子域 URL 正是一个高熵、不可猜测的能力引用，URL 轮换等价于使旧能力失效。
- **Bell-LaPadula 与最小权限**[7]：强调访问的「不向上/不向下」约束。mobile ai 遵循最小权限（least privilege）[8]：控制面只持有订单、码与绑定状态，**不持有任何业务数据的访问能力**。
- **Let's Encrypt 与 ACME**[9]：自动化证书签发是「边缘 TLS」普及的基础；mobile ai 复用 Cloudflare 为每个子域自动签发的证书，无需用户自备域名或证书。

3 系统架构（System Architecture）

3.1 分层总览

mobile ai 严格分为三层，**控制面与数据面在代码、部署与信任边界上完全隔离**。



关键事实：Cloudflare Tunnel 的 ingress **只按 hostname 匹配，不做 path/query 路由**。因此「多工具」由用户机器上的本地 mux（:5380 区段，path `/t/〈name〉` → `127.0.0.1:〈port〉`，含 WebSocket upgrade 透传）实现；隧道 ingress 恒为一条 `sub.newapi.email` → `http://127.0.0.1:〈muxPort〉`。

3.2 控制面（Control Plane）

v1.0 部署于 Node.js + SQLite（本地或 VPS），职责被刻意收窄为五类：

职能	机制（v1.0）
无码自动注册（L1）	machine code + platform 作为唯一身份标识；activate 幂等——同机重复激活返回既有凭据，无需一次性码
机器绑定 + 心跳（T1）	单终端策略：一台授权机器；30s 心跳，追踪 `cumulative_online_ms`（5min delta cap）与生命周期
工具管理（T2）	每绑定可挂载多个 `{name, port}` 工具；admin 侧实时可见，本地 mux 路由表同步
推荐归因（X9）	activate 时可选 `ref` 参数记录 referrer machine code；admin stats 聚合 Top N

URL 轮换	整条隧道换随机子域，旧 URL 立即失效、新 URL 数秒生效；机器码不变、客户端无需重跑

控制面**不代理任何业务流量**——这是零留存论证（§4.3）的基础。

v0.5 → v1.0 演进：v0.5 控制面部署于 Cloudflare Workers + D1（边缘计算），使用 magic-link 邮件登录与一次性认证码；v1.0 将控制面迁移至 Node.js + SQLite（降低冷启动延迟、简化部署），取消一次性码机制，以 machine code 直接作为身份标识实现无码自动注册。安全语义不变（URL-as-credential + 机器绑定），但用户摩擦进一步降低：从「申请码 → 邮件接收 → 填码激活」简化为「一条命令即完成」。

3.3 数据面与本地 mux

- **cloudflared（token 模式）**：`tunnel --token 〈T〉run`，ingress 由控制面经 CF API 写入；本地无 `config.yml`。
- **mux**：daemon 内固定端口 HTTP 反代，路由表如下（本地内存 + `state.json`）：
  - `GET/POST /` → 默认工具（DSH :3080）
  - `/t/〈name〉/…` → `tools[name].addr`（与 query `?tool=〈name〉` 等价，mux 归一为 path）
  - `/t/〈name〉/ws`、upgrade → WebSocket 透传（含 DSH Origin 剥离）
  - 未知 name → `404 {error:'tool not found', tools:[…]}`（排障友好）
- **多工具零成本**：新增一个工具只改本地 mux 路由表，**不建第二条隧道、不加 CNAME**；URL 轮换一次覆盖全部工具。

3.4 TLS 终结位置与端到端加密

TLS 在 **Cloudflare 边缘终结**，随后经 Cloudflare 与用户机器之间加密的隧道通道转发至本地 mux。对业务流量而言：平台**不解密、不缓存、不留存**；边缘仅做 TLS 终结与字节转发。这与「在用户机器上终结、全程端到端」的 VPN 方案不同——mobile ai 信任 Cloudflare 边缘作为**诚实转发者（honest forwarder）**，换取消费端零安装与平台侧极简。

4 安全模型（Security Model）

4.1 URL-as-Capability：URL 即能力引用

随机子域（高熵、不可枚举）是**访问的唯一凭证**。这正符合能力安全范式 [5][6]：一个不可伪造的引用本身即授权，持有即可访问。其安全强度取决于子域的熵与不可猜测性（`newapi.email` 下随机子域，攻击者无法枚举有效值）。**撤销语义**：URL 轮换 = 生成新能力引用并使旧引用不可再解析（DNS/ingress 指向变更），等价于 capability revocation——这是传统会话 token「无法主动作废」所不具备的性质。

4.2 无码自动注册与机器绑定（v1.0）

- **身份 = machine code**：`MAI-XXXXX-XXXXX` 格式，由客户端首次启动时生成并持久化于 `state.json`。activate 请求携带 machine code + platform，控制面据此幂等注册/返回既有绑定——**无需一次性码、邮件或人工确认**。

- **幂等语义**：同 machine code 重复 activate → 返回既有 tunnelToken + URL（不创建新隧道）；不同 machine code → 新建绑定。
- **机器绑定**：单终端策略锁的是「你的电脑」（一台授权机器）；手机/平板同时打开同一 URL 属**客户端数量**，不受限。错机心跳自动吊销（「错机即杀」）。
- **纵深**：URL 能力 + machine code 绑定 + 心跳吊销构成三层，任一层的实现缺陷都被其余层兜底。
- **v0.5 对比**：一次性码在 v1.0 中被移除。安全分析：码的威胁模型是「邮件泄露 → 他人代为激活」，但 machine code 本身已绑定硬件指纹（platform + hostname hash），等价于将「码泄露窗口」压缩为零——身份即机器，无需中间凭据。

4.3 零留存论证（Zero-Retention）

命题：mobile ai 控制面在稳态运行下不持有、不解密、不留存用户的任何业务流量内容。

论证要点：(1) 数据面路径为「手机 ↔ Cloudflare Edge ↔ cloudflared（用户机器）」，控制面 Workers 不在该路径上；(2) ingress 仅含 hostname→ 本地地址映射，无流量内容字段；(3) D1 中 `bindings` 表仅存订单、码状态、机器码哈希与子域，**无请求/响应体**。故平台对业务流量是零知识的（zero-knowledge control plane）。边界声明：暴露什么服务由用户负责，平台不做内容审查（见 `SECURITY.md` 已知边界）。

4.4 威胁模型与信任假设

- **诚实转发者**：Cloudflare 边缘被假设为不主动解密/留存业务流量（§3.4）。
- **机器码为实用级**：是威慑 + 心跳撤销，非 DRM；硬件变更需人工重绑（`SECURITY.md`）。
- **URL 泄露**：等价于密码泄露，缓解 = 一键轮换 +（可选）Cloudflare Access 邮箱验证第二层。

5 容量分析（Capacity Analysis）

`docs/CAPACITY.md` 尚不存在，以下基于 **Cloudflare 免费计划公开限制**做保守估算。假设：单用户、默认 DSH（:3080）+ 1∼2 个附加工具，交互式使用。

资源	免费计划公开限制（保守取值）	mobile ai 消耗点	估算与结论
Workers（控制面）	∼10 万 req/天、每请求 CPU ≤10ms（免费档）	magic-link / 码核销 / 心跳 / 轮换，**低频事件驱动**	单用户每天数十 ∼ 数百次控制请求 ≪10 万；瓶颈仅在多租户聚合
D1（控制面存储）	5 GB / 库	`bindings`、码状态、邮件队列元数据，**无流量内容**（§4.3）	单用户 KB∼MB 级，5GB 极宽裕；多租户按绑定数线性增长
Cloudflare Tunnel（数据面）	无官方并发上限;带宽受边缘网络约束	业务流量全量经隧道转发（TLS 终结后）	**非配额瓶颈**；受用户上行带宽与 CF 边缘转发限速约束，交互式 DSH（文本/WS）远低于任何硬上限

保守结论与假设说明：

1. **控制面配额对单用户不构成约束**。Workers 的 ~10 万 req/天面向**全平台聚合**；单用户的控制事件（心跳 30min ≈ 48/天 + 偶发登录/轮换）占比可忽略。真正需要监控的是**多租户聚合**下 Workers 请求数与 D1 写入速率。
2. **数据面不受免费配额硬顶**，但受物理带宽约束：DSH 这类以文本/JSON/WebSocket 为主的负载上行占用极小；大文件下载受用户**家庭上行带宽**（典型 20–100 Mbps）而非 CF 配额限制。
3. **假设**：(a) 用户为交互式使用，非高吞吐文件同步；(b) 免费计划限制以 Cloudflare 官方文档为准，可能随时间调整——本文取保守值；(c) WS/SSE 长连接占用的是**并发连接数**而非请求配额，CF Tunnel 无官方并发上限故不计入。
4. **单点风险**：mux 为本地单点（挂 = 全工具不可用，但隧道存活、心跳照发），由 daemon KeepAlive 重启恢复——这是可用性而非容量问题。

与 VPN 方案的资源对比：Tailscale/ZeroTier 在两端各维护一个常驻 agent（WireGuard 内核态或用户态转发、DERP relay），其资源开销随「设备数 × 活跃连接」增长；mobile ai 的数据面开销集中在**单一用户机器的 cloudflared + mux**，消费端（手机浏览器）仅占用普通 HTTPS 连接资源。因此 mobile ai 的容量瓶颈是**每用户一条隧道 + 控制面事件速率**，而非全网设备矩阵——这对「个人访问自家一台机器」的场景是更贴合的资源模型。

建议：上线后以 `dns-probe`（美国 runner E2E）+ `/mai/healthz` 持续观测控制面延迟与隧道重连时延，作为容量假设的实证校准。

6 评估：设计决策 Trade-off（Evaluation）

#	决策点	备选方案	采纳	收益 / 代价（trade-off）
D1	身份认证方式（v0.5→v1.0）	A v0.5 一次性码 / B magic-link+ 长效 token / C **v1.0 machine code = identity（无码）**	**C**	[OK] 零摩擦、零邮件依赖、幂等；[!] machine code 泄露 = 他人可代为激活（缓解：机器绑定 + 心跳吊销）
D2	多工具拓扑	a 每服务一条隧道 / b edge Worker path 路由 / c **本地 mux**	**c + ?tool=**	[OK] URL 轮换从 N×2 API→1 次；启停秒级可逆；WS/SSE 已验证；[!] mux 单点
D3	URL 形式	?tool= query vs 子域前缀	**?tool=**	[OK] 零 DNS、零新增 CNAME，轮换 = 换 sub；[!] URL 略长（可接受）
D4	TLS 终结位置	边缘终结 vs 用户机器端到端	**边缘终结**	[OK] 消费端零安装、平台极简；[!] 引入「信任 CF 边缘为诚实转发者」假设
D5	撤销机制	会话 token（不可主动作废）vs **URL 轮换 = 能力失效**	**URL 轮换**	[OK] 主动撤销语义清晰、用户自助；[!] 需客户端无需重跑（已满足：机器码不变）

D6	鉴权层数	仅 URL vs **URL + （可选）CF Access**	**分层，Access 可关**	[OK] 纵深防御；[!] Access 邮箱验证对手机首访多一步（可关闭）
D7	机器绑定强度	DRM 级 vs **实用级（威慑 + 心跳撤销）**	**实用级**	[OK] 实现简单、误伤可控；[!] 硬件变更需人工重绑（非 DRM）
D8	消费端形态	原生 App vs **系统浏览器**	**浏览器**	[OK] 零安装、跨平台；[!] 受浏览器能力约束（无本地通知等）

评估结论：所有决策都在「用户摩擦」与「安全强度/平台复杂度」之间做显式取舍，且**每一层的安全动作都被设计为廉价可重复**（轮换 = 一次点击、启停 = 秒级），因为「安全动作越便宜，用户越会用」。值得注意的是 D2 与 D4 的耦合：选择本地 mux（D2）使得 URL 轮换能一次覆盖全部工具，而边缘 TLS 终结（D4）使该轮换只需改 DNS/ingress、无需触碰数据路径——两项决策相互放大，共同把「撤销」这一安全动作的成本压到最低。反之若采用每服务一条隧道（D2-a），轮换成本随工具数线性增长，安全动作将因昂贵而被用户回避，反而削弱整体安全性。这说明**撤销成本是安全模型的一等设计变量**，而非事后补救。

7 讨论与局限（Discussion & Limitations）

1. **信任边界的转移而非消除**：mobile ai 把「两端装客户端」的摩擦换成「信任 Cloudflare 边缘为诚实转发者」。对高敏感场景（医疗、金融），应叠加 CF Access 第二层或改回端到端 VPN。
2. **机器码非 DRM**：实用级绑定可被虚拟机指纹克隆绕过，依赖心跳撤销兜底；这是刻意选择——DRM 级绑定带来的误伤（换硬盘/主板）对个人用户不可接受。
3. **单终端策略的粒度**：锁「一台授权机器」而非「一个会话」，多设备消费同一 URL 不受限；若需按消费者授权，需引入消费者侧身份（当前刻意不做以保零安装）。
4. **容量为估算**：§5 基于公开免费配额的保守假设，未含实测 P99；需 `dns-probe` 与生产遥测校准。
5. **mux 单点**：本地 mux 故障使全工具不可用（隧道存活），靠 KeepAlive 恢复；对多租户 SaaS 化需考虑健康检查与自动降级。
6. **URL 熵假设**：撤销语义依赖子域不可枚举；若未来引入可预测命名（如用户自定义前缀），需重新评估能力强度。当前随机子域由控制面在激活时生成，其熵应保证暴力枚举不可行；本文将其作为安全公理而非实测结论。
7. **单用户假设**：全文容量与绑定分析以「一个账号、一台授权机器」为基线；多租户聚合下的配额治理（§5 第 1 点）与按消费者授权（局限 3）是 SaaS 化时必须补全的维度，超出本文范围。

8 结论（Conclusion）

mobile aiv1.0 证明了「零安装 + 消费端零应用 + 平台零业务流量接触 + 无码自动认证」四者可以共存：通过**控制面/数据面严格分层、本地 mux 实现一条隧道多工具**，以及以**能力范式**为核心的安全模型（URL-as-credential、轮换 = 撤销、machine code 绑定 + 心跳吊销纵深）。系统在 v1.0 已投入生产，支持 L1（无码认证）、L2（本地 mux 多工具）、T1（在线时长追踪）、T2（工具预设与 UX）、A1（别名）、X9（推荐归因）六项核心功能。本文的分析表明，面向个人计算设备的远程访问不必在「客户端摩擦」与「可撤销的安全凭证」之间二选一——关键在于把**身份管理与撤销语义**放在控制面、把**数据与算力留在用户机**

器。未来工作：消费者侧可选身份层（在不破坏零安装前提下）、实测容量校准、多租户聚合配额治理、以及 CF Access 第二层鉴权的标准化集成。

参考文献（References）

[1] Tailscale. *Tailscale: WireGuard-based zero-config mesh VPN*. https://tailscale.com/blog （官方技术博客，含 WireGuard 与 DERP relay 机制说明）.

[2] ZeroTier. *ZeroTier Network: Software-defined virtual LANs*. https://www.zerotier.com/ （官方文档，虚拟局域网语义）.

[3] ngrok, Inc. *ngrok Documentation: secure inbound tunnels*. https://ngrok.com/docs （官方文档，隧道生命周期与身份）.

[4] Cloudflare, Inc. *Cloudflare Tunnel (cloudflared) Documentation*. https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/ （含 token 模式、ingress hostname 匹配与 quick tunnel）.

[5] M. D. McLean, *Capability-based computer security*, Communications of the ACM 36(6), 1993. （能力安全经典综述）

[6] R. M. Needham and D. J. Wheeler, *Capability-based systems: a survey*, ACM Computing Surveys 29(4), 1997. （能力系统综述，含撤销语义）

[7] D. E. Bell and J. P. LaPadula, *Secure computer system: Mathematical models and classification*, MTR-2547, MITRE Corporation, 1973. （Bell-LaPadula 模型）

[8] J. L. Hartman, *The principle of least privilege*, IEEE Computer 27(8), 1994. （最小权限原则）

[9] B. A. Abrahamsen et al., *Automatic Certificate Management Environment (ACME)*, RFC 8555, IETF, 2019. （Let's Encrypt 所依赖的证书自动化协议）

[10] Cloudflare, Inc. *Cloudflare Free Plan limits (Workers requests/day, D1 storage)*. https://developers.cloudflare.com/workers/platform/limits/ 与 …/d1/limits/ （免费计划公开限制，§5 容量估算依据）.

[11] Cloudflare, Inc. *Cloudflare Access (Zero Trust) Documentation*. https://developers.cloudflare.com/cloudflare-one/policies/access/ （邮箱验证策略，§4.2 第二层鉴权）.

本文为系统设计与分析论文；系统实现见仓库 *README.md*、*docs/GUIDE.md*（*v1.0* 现状）与 *docs/DESIGN-lite.md*（极简版工程设计）。